K4 CONNECT MANAGER

k4cm Guide

Sharing the mh01.net Elecraft K4s with members operating from home

---

Admin · User · Calendar · Internals

16 October 2026

Contents

PART 1 Overview

1.1 A session, end to end
1.2 Where to read next

PART 2 Admin guide

2.1 Install and initialise
2.2 Describe the radios
2.3 Router and firewall
2.4 Policy settings
2.5 Day-to-day administration
2.6 Operations

PART 3 User guide

3.1 Before you start
3.2 Install and set up
3.3 Operating
3.4 Reading `k4cm status`
3.5 The end of a session
3.6 Queues
3.7 Connecting by hand
3.8 Troubleshooting
3.9 Where things are kept

PART 4 Calendar guide

4.1 Booking rules
4.2 Modes
4.3 When the booking starts
4.4 From the command line: `k4cm schedule`
4.5 In the browser: `k4cm calendar`

PART 5 Internals guide

5.1 Security model
5.2 Race-free allocation
5.3 Bookings engine
5.4 The monitor
5.5 Where state lives
5.6 Server layout
5.7 Client layout
5.8 CAT layer
5.9 Testing
5.10 Conventions

PART 1

Overview

What k4cm is, and how a session runs from request to release.

k4-connect-mgr (k4cm) lets club members borrow the Elecraft K4s at a central site (mh01.net) from their own K4 or K4/0 at home. One Python package ships two programs:

Program	Runs on	Does
`k4cm-server`	The Windows PC at the K4 site	FastAPI service over HTTPS. Tracks radios, users, sessions, queues, bookings and the audit trail in SQLite. The only component that talks CAT to the central K4s. Relays each radio's remote link.
`k4cm`	Each member's Windows, macOS or Linux machine	Typer CLI. Requests a radio, drives the member's own panel over CAT, watches the session, books radios ahead, and serves the browser calendar.

Access is enforced by the radios themselves. Each session sets a fresh random password on the K4, and every ending replaces it with another, so an old password never opens a session.

1.1 A session, end to end

1. `k4cm request k4-1` asks the server for that radio by name. There is no pool: each radio has its own queue.
2. The server claims the radio in one atomic database step, sets a new password on it (`RRP<pw>;` + `RRC1;`) and returns the password and public address.
3. The client tells the member's panel to connect (`RRT<host>:<port>?<pw>;`). The panel dials the server's relay port, which passes the link to the radio.
4. The client's watcher polls the session, warns before the deadline and offers accept, extend or release.
5. On any ending (release, expiry, reclaim, force-release) the server cuts the relayed link, sets another random password, sends `RRC0;`, and offers the radio to the next member in its queue.

1.2 Where to read next

- **Admin guide**: installing and running the site server.
- **User guide**: taking, extending and releasing a radio.
- **Calendar guide**: booking radios ahead, by CLI or in the browser.
- **Internals guide**: how the parts work, for anyone changing the code.

PART 2

Admin guide

Installing, configuring and running the site server.

The site server runs on one Windows PC on the radios' LAN. You install it once, describe the radios in `server.toml`, forward ports on the router, and issue members their API keys.

2.1 Install and initialise

1. Run `k4cm-server-setup.exe` from the Releases page. It installs to `Program Files\K4 Connect Manager`, adds the VC++ runtime if missing, and registers two Task Scheduler jobs: start at boot, and a watchdog that probes `/health` every 5 minutes and relaunches the server if it doesn't answer.
2. Run `k4cm-server init`. It creates `server.toml` and the database, generates a self-signed TLS certificate, prints its SHA-256 fingerprint, and creates the admin account, printing its API key once.
3. **Publish the fingerprint to members.** Their client asks them to confirm it at setup.

Site data lives in `C:\ProgramData\k4cm\`: `server.toml`, `k4cm.db`, `tls\`, `logs\`, `backups\`. Reinstalls and upgrades never overwrite it.

Command (on the server PC)	Does
`k4cm-server init`	First-time setup, as above
`k4cm-server run`	Run the server in the foreground (disable the watchdog task first)
`k4cm-server backup`	Take a database backup now
`k4cm-server fingerprint`	Print the certificate fingerprint again
`k4cm-server reset-admin`	Issue a new admin key. Deliberately local only: there is no remote path

2.2 Describe the radios

Each radio is a `[[radios]]` entry in `server.toml`. `k4cm admin radios --reload` picks up changes without a restart.

```
[[radios]]
id = "k4-1"
name = "K4-1"
host = "10.0.0.11"          # LAN address for CAT, never given to members
cat_port = 9200
remote_port = 9204           # the radio's own remote-link port
relay_port = 27201           # port on THIS server that relays to remote_port
public_host = "mh01.net"     # where members' panels dial
public_port = 27201          # defaults to relay_port
enabled = true

  [radios.power]             # power-on needs a PDU or relay
  method = "http"            # or "command", or "none"
  url = "http://10.0.0.50/api/outlet/3/on"
  credentials_ref = "K4CM_PDU_MAIN"   # env var holding user:password
```

Powering off is CAT (`PS0;`). Powering on cannot be done in software, because a K4 accepts no commands while off and has no Wake-on-LAN, so `[radios.power]` points at a PDU or relay.

2.3 Router and firewall

- Forward the API port (8443) to the server PC.
- Forward each radio's public port to **the server PC's `relay_port`, not to the radio.** The relay is what makes the end of a session stick: a K4 keeps an attached panel when its password changes.
- **Never forward a CAT port.** CAT has no authentication; anyone reaching it can read the password with `RRP;` or power the radio off.
- Over a VPN, set `public_host` to the server PC's LAN address instead. The installer's firewall rule covers the API and every relay port.

A radio without `relay_port` is reached directly. That works, but an ended session then only stops new connections; a connected member stays on until they disconnect.

2.4 Policy settings

Section · setting	Default	Meaning
`[session] max_session_minutes`	240	Length of a new session
`[session] warn_period_minutes`	15	How early the member is warned
`[session] ack_grace_minutes`	5	Extra time after the deadline if the member doesn't answer
`[session] extension_minutes`	60	What one extension adds (only when nobody is queued)
`[session] queue_hold_minutes`	10	How long a freed radio is held for the next in queue
`[schedule] hold_minutes`	10	How long a booking waits for its owner before others may borrow
`[schedule] ending_notice_days`	30	When owners are reminded a series is ending
`[monitor] poll_interval_seconds`	30	Monitor tick

Section · setting	Default	Meaning
`[monitor] disconnect_grace_minutes`	5	Reclaim a radio whose panel has been gone this long
`[monitor] cat_timeout_seconds`	5	CAT and relay connect timeout
`[relay] bind`	0.0.0.0	Address relay listeners bind
`[relay] check_address`	true	Admit a panel only from the member's last API address
`[backup] interval_hours` / `retention`	24 / 14	Backup cadence and copies kept

2.5 Day-to-day administration

Admin commands run from any client configured with the admin key.

Command	Does
`k4cm admin users`	List accounts
`k4cm admin users --add W1AW`	Create an account, printing its API key once
`k4cm admin users --reset-key W1AW`	Replace a lost key; the old one stops working
`k4cm admin radios`	Radios and reachability
`k4cm admin radios --reload`	Re-read the `[[radios]]` section
`k4cm admin radios --power k4-1 --action cycle`	Power control
`k4cm admin sessions`	Recent and active sessions
`k4cm admin sessions --force-release 42`	End someone's session
`k4cm admin audit --hours 24`	Read the audit trail
`k4cm admin service k4-1 --start "2026-10-20 09:00" --duration 4h`	Mark a radio "Being serviced"
`k4cm schedule add ... --override`	Book over members' bookings; they get a notice

2.6 Operations

- **Logs**: `C:\ProgramData\k4cm\logs\k4cm-server.log`, rotated. A failed start is logged as `cannot start: ...`. `watchdog.log` records each restart.
- **Backups**: nightly by SQLite's online backup API, safe while running. Point `[backup] directory` at a share for off-machine copies. To restore, stop the scheduled task, replace `k4cm.db`, start it again.
- **Health**: `GET /health` needs no authentication and exposes no data.
- **Certificate rotation**: every client fails loudly until its member re-runs `k4cm config`. Rotate deliberately and announce the new fingerprint.

- **Diagnosing dropped links**: `k4cm admin audit` records each relayed link's end as `relay.disconnect`: how long it lasted, which side closed it first, and the bytes and last data from each side. A K4 hangs up on a client silent for about 10 s, so "radio closed the link" with the member's last data about 10 s before means the member's internet path died; the radio closing while the member was still sending points at the radio or the site LAN. `relay.unreachable` means the server could not reach the radio's remote port at all.
- **Relay latency**: `scripts/relay_latency.py` on the server PC measures what the relay adds (about 0.25 ms per round trip on a Mac).

PART 3

User guide

Taking, extending and releasing a radio from home.

Members install the client, run `k4cm config` once, then `k4cm request <radio>` to operate. You never type a radio password and never connect your panel to the K4s directly.

3.1 Before you start

- **Callsign and API key** from the administrator. Treat the key like a password.
- **The server's certificate fingerprint**, also from the administrator.
- **Your panel's IP address and CAT port** (normally 9200). Give it a DHCP reservation so it doesn't change.
- **Computer and panel on the same internet connection.** The station admits a panel only from the address your computer uses to reach the server, so a hotspot or work VPN won't work.

3.2 Install and set up

1. Install `k4cm-client-setup.exe` (Windows), `k4cm-client-<version>.pkg` (Mac) or the Linux tarball from the Releases page. Unsigned-installer warnings are expected; choose Keep / Run anyway, or Open Anyway in macOS Privacy & Security.
2. Open a new terminal and run `k4cm config`. Answer: server URL (`https://mh01.net:8443`), callsign, API key, panel address, panel CAT port.
3. Compare the fingerprint it shows with the published one, character for character, and answer `y` only if they match. That certificate is pinned and becomes the only one k4cm accepts.
4. Optional: `k4cm --install-completion` for Tab completion.

Windows may warn because the installer is not code-signed. In Edge choose ⋯ → **Keep** → **Show more** → **Keep anyway**, and when it runs **More info** → **Run anyway**. On a Mac, if the package is blocked, open **System Settings** → **Privacy & Security** and click **Open Anyway**. Installing a newer version over an older one keeps your settings.

`k4cm config` ends with `Server reachable: k4cm <version> at https://mh01.net:8443`. Run it again any time to change an answer; it offers your current values as defaults. If you leave the panel address blank, k4cm still works but prints the connection details for you to enter on the panel (see Connecting by hand).

On a Mac using bash rather than the default zsh, Tab completion also needs this once, because Terminal's login shells never read `~/.bashrc`:

```
echo '[ -f ~/.bash_completions/k4cm.sh ] && . ~/.bash_completions/k4cm.sh' >> ~/.bash_profile
```

3.3 Operating

Command	Does
`k4cm status`	Every radio's state, time left, queue length, reachability and next booking, plus your session and panel
`k4cm request k4-1`	Take the radio, connect your panel, and keep watching the session
`k4cm request k4-1 -d 90m`	Ask for a shorter session than 4 hours
`k4cm request k4-1 --queue`	Join its queue without asking if it is busy
`k4cm request k4-1 --no-watch`	Connect and return to the prompt
`k4cm request k4-1 --no-connect`	Take the radio and print the details to enter on the panel by hand
`k4cm watch`	Pick up watching again; also connects auto-session bookings
`k4cm release`	Disconnect and hand the radio back
`k4cm queue` / `k4cm queue --leave`	Your place in the queue / leave it

After `request`, k4cm waits up to 12 seconds for the station to confirm your panel arrived. If it doesn't, the radio is handed straight back and k4cm says why. You can hold one session at a time.

A successful request looks like this. Times are shown in your computer's time zone. If someone has booked the radio for later, your session ends when their booking starts, and k4cm tells you so up front.

```
K4-1 is yours until Sat 10 Oct 2026 18:15.
  Time allowed: 240m
Local radio: connected to mh01.net:27201

Watching your session — Ctrl-C to stop (your session keeps running).
```

3.4 Reading `k4cm status`

```
RADIO       STATE                    UNTIL      QUEUE  REACHABLE  BOOKED
k4-1        in use by W1XYZ          72m        0      yes        next: K2ABC Tue 20 Oct 19:00
k4-2        free                     -          0      yes        now: W1AW to 21:00

You have no session and are not queued.
Your local radio: idle (not connected)
```

Column	Meaning
STATE	`free`; `in use by <callsign>`; `held for <callsign>` (freed and offered to the next in its queue); or `disabled` (out of service)
UNTIL	How long the current session has left
QUEUE	How many members are waiting for that radio
REACHABLE	Whether the server can talk to the radio. `NO` = off or a problem, tell the administrator; `?` = not checked yet
BOOKED	The booking running now, or the next one

3.5 The end of a session

Leave the watcher running. Fifteen minutes before the end it beeps and asks:

- **A**ccept: operate until the deadline, then the panel is disconnected.
- **E**xtend: add an hour, if nobody is queued and no booking starts within it. Otherwise treated as Accept.
- **R**elease: end now.

If you don't answer, you get 5 minutes past the deadline before the radio is taken back. If your panel stays disconnected for 5 minutes, the server ends the session. Ctrl-C stops watching but leaves the session running.

Whatever ends a session, the station disconnects your panel within a few seconds and changes the radio's password.

3.6 Queues

Each radio has its own queue. Joining tells you your place and the latest time the radio could reach you, if everyone ahead uses their full time. If a booking starts right when the radio would reach you, you aren't queued, since you would get no time on it. When it's your turn the radio is **held for you for 10 minutes** and the watcher asks `Take it now? [Y/n]`. No answer in 10 minutes passes it to the next member, so keep `k4cm watch` running and stay near the computer.

3.7 Connecting by hand

If you set up k4cm without a panel address, or used `--no-connect`, k4cm prints the details instead:

```
Connect your panel manually with these details:
  Host:     mh01.net
  Port:     27201
  Password: Xq3...
```

Enter them in your panel's remote-connection settings and connect. The password works for this session only, and the station still admits only the address your computer is using. Use **Disconnect** on the panel and then `k4cm release` when you're done.

3.8 Troubleshooting

Message	What to do
`config is incomplete ... Run 'k4cm config'`	k4cm isn't set up on this computer, or a setting is missing. Run `k4cm config`
`invalid or missing API key`	The key is wrong or has been replaced. Paste the current key with `k4cm config`. Repeated bad keys make the server answer more and more slowly for a while
`certificate does not match`	If the administrator announced a new certificate, run `k4cm config`, compare the fingerprint and accept it. If nobody announced a change, don't accept it; tell the administrator
Server can't be contacted	Check your internet connection and that the URL is `https://mh01.net:8443`. If it still fails, the station may be down
`cannot talk to your local radio at 192.168.x.x:9200`	Check the panel is on, on your home network, at the address you gave `k4cm config` (it may have changed), CAT port 9200. The radio is handed back automatically; request it again once the panel answers
`your radio did not connect ... within 12 seconds`	The station never saw the panel arrive: panel and computer reach the internet differently (VPN, hotspot, a second router); outbound ports 27201 and 27211 to mh01.net are blocked; or the central radio is off (check REACHABLE in `k4cm status`)
`<radio> is disabled by the administrator`	The radio is out of service. Try the other one
`you already have ... assigned; release it first`	`k4cm status` shows it; `k4cm release` ends it
`Your local radio: connected (or connecting) to a remote K4` with no session	The panel is still linked from an earlier attempt, or connecting elsewhere. Press **Disconnect** on the panel
Windows: `'k4cm' is not recognized`	Open a new Command Prompt or PowerShell window; Windows only sees the new program in windows opened afterwards

Anything else: tell the administrator what you typed and the exact message k4cm printed.

3.9 Where things are kept

	Windows	Mac / Linux
Settings and API key	`%APPDATA%\k4cm\config.toml`	`~/.config/k4cm/config.toml`
Pinned server certificate	`%APPDATA%\k4cm\server-cert.pem`	`~/.config/k4cm/server-cert.pem`
This guide	Start menu: **K4 Connect Manager Client → K4CM Guide**	`/usr/local/share/doc/k4cm/k4cm-guide.md`

The settings file holds your API key: don't share it or copy it onto other people's computers. Uninstalling leaves the settings in place; delete the folder yourself if you want them gone.

PART 4

Calendar guide

Booking radios ahead, from the command line or the browser.

Members can book a radio ahead, once or on a repeating pattern, from the CLI (`k4cm schedule`) or in the browser (`k4cm calendar`). Times are entered in your local zone, and a weekly 19:00 stays 19:00 across daylight-saving changes.

4.1 Booking rules

- **Overlaps are refused.** A single booking that clashes is rejected. A series is booked with the clashing dates skipped, and you're told which and whose.
- **Every series ends**, by `--count` or `--until`, and nothing may be booked more than a year ahead.
- **The radio is held for its owner** for the first 10 minutes (`[schedule] hold_minutes`); others are refused.
- **After the hold, others may borrow it**, warned that the owner can take it back. When the owner requests it, the borrower gets the usual warning and ack grace, and the owner goes to the front of the queue.
- **Every other session ends at the next booking**, and an extension is refused if a booking, a "Being serviced" block or a queue is in its way.
- **Admins** can mark a radio "Being serviced" or book with `--override`; members' clashing dates are removed and they get a notice.
- Owners are reminded `ending_notice_days` (30) before a series' last date.

4.2 Modes

Mode	At the start time
`--mode reserve` (default)	The radio is held for you. Run `k4cm request` to connect
`--mode auto`	The server opens the session itself. A running `k4cm watch` connects your panel; without one, `k4cm request` takes it over

The server cannot reach a member's panel, which is why an auto-session needs the watcher running on your machine before the booking starts.

4.3 When the booking starts

1. For the first 10 minutes the radio is held for you, and anyone else who asks is refused.
2. If you haven't connected by then, someone else may borrow it. They are warned that you can take it back.

3. If you then run `k4cm request`, the borrower gets the usual warning and grace period, and the radio comes to you. Keep the watcher running to accept it.
4. Your session lasts until the booking ends. You can extend it like any session, if nobody is queued and nothing else is booked in that hour.

About a month before a series ends, k4cm reminds you and shows how to extend it. If an administrator changes or removes one of your bookings, or marks the radio **Being serviced**, you see a notice the next time you use k4cm.

4.4 From the command line: `k4cm schedule`

```
k4cm schedule add k4-2 --start "2026-10-20 19:00" --duration 2h \
     --repeat monthly --on 3rd-tue --until 2026-12-31
```

k4cm always lists the dates, the last one, and any skipped ones, and asks before booking.

`--repeat`	`--on` **examples**
`daily`	(none)
`weekly`	`tue,thu`
`monthly`, by date	`1,15` or `last` (dates 1–28, or the last day)
`monthly`, by weekday	`3rd-tue`, `2nd-sat,4th-sat`, `last-fri`
`yearly`	(none)

`--every 2` means every second day, week, month or year.

Command	Does
`k4cm schedule list`	Everyone's bookings for two weeks. `--mine`, `--member CALL`, `--radio`, `--from` / `--to`
`k4cm schedule show 12`	One booking or series, every date, any skipped
`k4cm schedule edit 12 --start "..."`	Change it. For a series: `--scope this --date D`, `--scope following --date D`, or `--scope all` (default)
`k4cm schedule edit 12 --until 2027-03-31`	Extend or shorten a series, keeping its pattern
`k4cm schedule skip 12 --date 2026-11-17`	Cancel one date of a series
`k4cm schedule remove 12`	Delete. `--scope` and `--date` as for edit

Only the owner or an administrator can change a booking.

4.5 In the browser: `k4cm calendar`

`k4cm calendar` serves the calendar on 127.0.0.1 and opens it in your browser. Leave the terminal open; Ctrl-C closes it. `--port` picks the local port and `--no-browser` prints the address instead.

- **Week** view shows one radio, chosen from the list. **Day** view shows every radio side by side. Click a day or radio heading to switch.
- **Drag across empty time** to book it. The form offers every repeat option and previews the dates and any skips.
- Your bookings are solid; others' are pale with the owner's callsign; "Being serviced" is striped.
- **Click a booking** for details, and edit or delete your own. **Drag one of yours** to move it; for a series choose this date, this and later, or all.

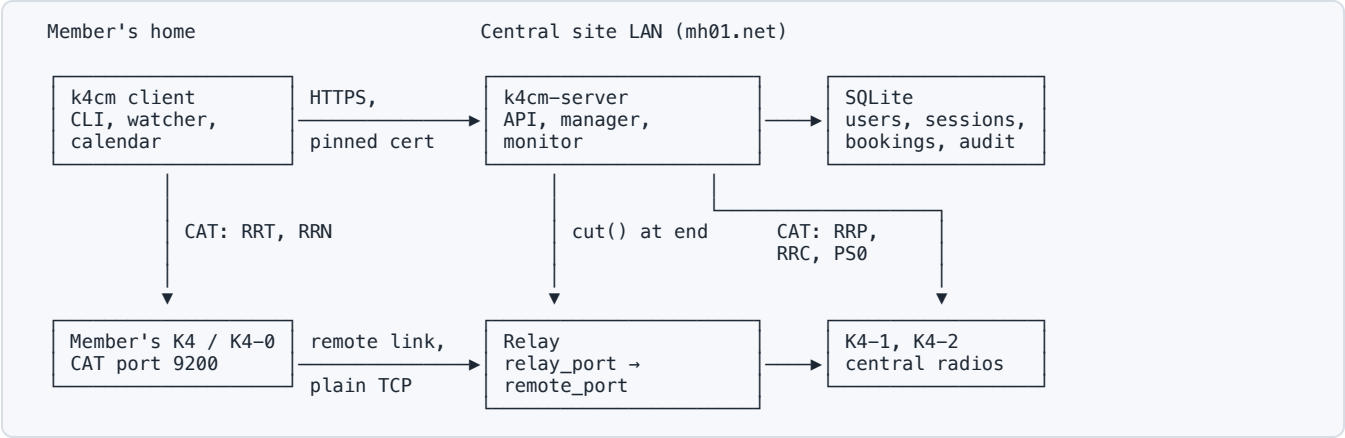
The page talks to the server through the `k4cm` process, which adds your API key and trusts only the pinned certificate, so the browser never sees the key. A read-only iCalendar feed is designed but deferred: Google Calendar would need the server reachable with a publicly trusted certificate.

PART 5

Internals guide

How the parts work, for anyone changing the code.

The security model is the assignment model: there is no per-radio ACL, and every path that ends a session goes through `AssignmentManager.end_assignment`, which cuts the relay and locks the radio.



The client controls sessions over HTTPS; the panel's remote link runs through the relay, which the server cuts when a session ends. Only the server speaks CAT to the central radios. Only the API and relay ports are forwarded; CAT never is.

5.1 Security model

- `_apply_password` sets a fresh `secrets`-generated password (`RRP<pw>;` + `RRC1;`) when a session starts. `_lock_radio` sets another and sends `RRC0;` on every ending. Passwords are never derived from a previous value, and there is no configurable seed.
- **The lockout does not drop an attached client** (hardware-verified 2026-10-09). `RRP` / `RRC0` only stop new connections, and the Programmer's Reference (D12) has no server-side disconnect.
- So `server/relay.py` owns the path. For a radio with `relay_port`, the panel connects to the server, which pumps bytes to `host:remote_port`. `end_assignment` calls `relay.cut()` before the lockout, and the K4 drops the silent client in about 2 s.
- The relay admits only an ACTIVE session's member, from `assignments.client_addr`, which `note_client_addr` refreshes on every watcher poll. The remote link is plain TCP; no UDP was seen on a live K4/0.
- The client also sends `RRN;` at session end (`_end_of_session_disconnect`), but that is only cooperative.

5.2 Race-free allocation

1. `request_assignment` claims the radio with `db.claim_radio`, an INSERT of a `pending` row. Partial unique indexes allow one `pending`/`active` row per radio and per callsign, so the loser of a race gets `ClaimConflict`.
2. Under `manager.radio_lock(radio_id)` it sets the password, then `activate`s the claim. Every CAT sequence that changes a password runs under that lock, so a lockout and a new setup never interleave.
3. State changes are conditional on the expected prior state (`finish_assignment`, `set_expiry`). `end_assignment` re-reads the row under the lock and takes a `still_due` predicate, because callers act on snapshots that go stale across awaits.
4. `abandon_stale_claims` fails orphaned `pending` claims after `claim_timeout`.

5.3 Bookings engine

- `k4cm/schedule.py` (shared with the client) turns repeat options into RRULEs and expands them in the owner's zone (`bookings.tz`, naive `start`), then converts to UTC, so local times survive DST.
- Each occurrence is keyed by its local date (`occ_date`). `booking_exceptions` skip or move a date under that key.
- Occurrences are never stored. `BookingService` (`server/bookings.py`) recomputes them and caches against `db.bookings_rev`, which every booking write bumps.
- Runtime rules live in the manager: `request_assignment` refuses during maintenance or another owner's hold, gives an owner a session to the booking's end, reclaims a borrowed radio, and caps everything else at the next booking. `manager.enforce_bookings` moves deadlines (`session_deadline`), opens auto-sessions once (`occurrence_state`), and promotes queues after a hold.

5.4 The monitor

`Monitor.tick()` runs once per poll interval: it expires queue offers, enforces deadlines (an acked session ends at its deadline, an unacked one gets `ack_grace` more), reclaims radios disconnected past `disconnect_grace`, and calls `enforce_bookings`. It is a plain coroutine so tests can drive it with shortened timers. Reachability is in memory only (`manager.reachability`, `None` = not yet probed).

5.5 Where state lives

State	Where	Notes
Radios	`server.toml` `[[radios]]`	`ServerConfig.reload_radios()` re-reads that section at runtime
Users, assignments, queues, bookings, audit	SQLite (`server/db.py`)	One connection behind an `RLock`, WAL mode, `CREATE TABLE IF NOT EXISTS` on every open, no migration framework
Reachability, disconnected-since	Manager memory	Filled by the monitor
Client settings, pinned cert	`config.toml`, `server-cert.pem`	Never overwritten by installers

5.6 Server layout

- `app.py` is thin: routes authenticate with `auth.require_user` / `require_admin` and delegate to `AssignmentManager`.
- `manager.py` holds the business logic and is the only place radios are touched. It raises `ManagerError(status, message, **extra)`, which `create_app` turns into JSON.
- `create_app(..., background=False)` suppresses the monitor and backup loops for tests. The relay starts in the lifespan regardless.
- `power.py`: power-off is `PS0;`; power-on is a shell command or HTTP call to a PDU under `[radios.power]`.

5.7 Client layout

- `main.py`: Typer commands plus the session watcher (`_watch_loop`), which polls `/assignments/current` and handles the expiry warning and queue offers.
- **Trust is a pinned self-signed certificate.** `ServerApi._verify()` passes the saved PEM to httpx as the only CA. `_is_certificate_error` walks the exception chain, since httpx buries `ssl.SSLError` inside `ConnectError`. `check_transport` refuses plain `http://` except to localhost.
- `radio.py` drives the member's panel with `RRT...;` and `RRN;`. The panel's own `CC;` is no evidence: a K4/0 reads `CC-1` for about 10 s after any `RRT`, and refuses a wrong password silently. So `_connect` confirms with `GET /assignments/current/link`, which reads the central radio's client count, and fails after `CONNECT_TIMEOUT`.
- `calendar.py`: a 127.0.0.1 server passing calls through `ServerApi`. API calls need the per-run page token and our own Host header, which keeps other sites and DNS rebinding out. The page is a string in `calendar_page.py` so installers need no data files.

5.8 CAT layer

`cat.py` sends ASCII commands terminated by `;` over TCP. `CatClient.request` skips unrelated status broadcasts while waiting for its prefix. `send_commands` PINGs afterwards to confirm the radio consumed them; `power_off` doesn't, since the radio stops answering.

Command	Sent to	Purpose
`RRP<pw>;`	central K4	Set the remote-access password
`RRC<n>;`	central K4	Allow n remote connections (0 refuses new ones; attached clients stay)
`RRT<host>:<port>?<pw>;`	member's panel	Connect; success is silent, failure may be `RRT!;` or nothing
`RRN;`	member's panel	Disconnect
`CC;`	either	Client count: n ≥ 1 attached, -1 connected as a client, 0 idle
`PING;` → `PONG;`	either	Reachability
`PS0;`	central K4	Power down

5.9 Testing

- `tests/conftest.py` builds a whole site per test: three `FakeK4`s on ephemeral ports (two central radios, one panel), a generated `server.toml`, a real database, manager, monitor and app. The `api` fixture wraps the app in `httpx.ASGITransport` with `ADMIN` and `W1AW` created.
- `tests/fake_k4.py` implements `RRP`, `RRC`, `RRT`, `RRN`, `CC`, `PING`, `PS0`, including a real TCP link between two fakes, and models the hardware quirks (`silent_refusal`, `refused_cc_window`, `RRC0;` leaving clients attached). Extend the fake rather than mocking `cat`.
- `test_e2e.py` runs uvicorn in a thread and the CLI in worker threads. `test_relay.py` drives `manager.relay.sync()` directly.
- `tests/hardware/` checks the fake's assumptions against real radios; it is skipped unless `hardware.toml` exists.
- Run: `.venv/bin/python -m pytest` (about 6 s) and `.venv/bin/python -m ruff check .`. Tags matching `v*` run the suite and build the four installers in CI.

5.10 Conventions

- Datetimes are UTC-aware; use `models.utcnow()`, `isoformat()`, `parse_dt()`, and store ISO strings.
- Every state change writes `db.audit(actor, action, detail)` with a dotted action name (`session.expired`, `relay.connect`).
- User-facing errors say what to do next. API keys are echoed exactly once.
- Module docstrings carry the why behind the code.
- Installers and `write_default_config` never overwrite site data.